https://github.com/ding89852/contemporary-artificial-intelligence/tree/project2

实验二 - 搜索算法

龚晓博 10245501469

题目1

算法实现

实现了一个基于 A* 算法的迷宫求解器:从inputs/maze.txt中读入起点、终点和字符网格，输出从起点到终点的最短路径。

A*的状态: Coord = (row, col),Tuple[int,int]类型

g值: g_score[node],dict[Coord, int]类型 代表从起点到该状态的当前已知最小代价; g_score[start] = 0; 在扩展父节点时更新子节点,g值只会下降

```
g_score = {start: 0}
...
tentative = g_score[node] + 1
if tentative < g_score.get(neighbour, float("inf")):
    g_score[neighbour] = tentative
```

h值: heuristic(node)

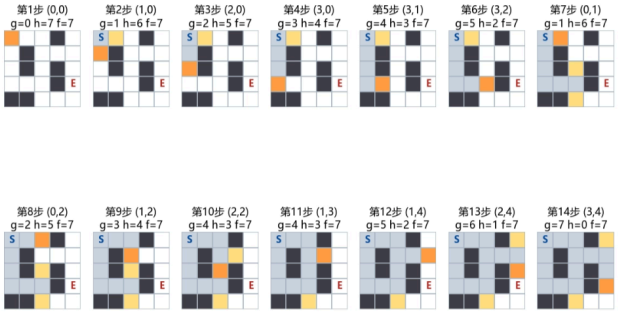
```
def heuristic(node: Coord) -> int:
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])
```

本题的启发式函数采用曼哈顿距离,具有可采纳性与一致性 可采纳:当前节点到终点的曼哈顿距离≤当前节点到终点的可行距离 一致性:曼哈顿距离满足三角不等式h(n)-h(m)≤cost(n,m) h(goal)=0

f值: f=g+h

算法步骤: ① set g_score[start] = 0, 加入open队列； ② 从open取出f值最小的节点； ③ 如果当前节点是终点,停止搜索并回溯路径； ④ 将该节点的邻居加入open队列,计算每个邻居的g值； ⑤ 对于每个节点,如果有更小的g值,更新g,f和父节点； ⑥ 当前节点进入closed队列,继续循环

最短路径形成过程:(蓝色为探索过的节点,橙色为当前节点,黄色为待展开的节点)



搜索效率评估

1. 不同启发函数的搜索规模与用时: (计时方法:使用`time.perf_counter()` 重复7次取中位数)

搜索策略	扩展节点数	生成节点数	队列峰值	路径步数	单次中位耗时	最优解占比	平均代价偏差
h = 0（Dijkstra 基线）	43.7	44.2	6.6	14.2	64.2 µs	21/21	0.000
曼哈顿距离 h	17.8	27.1	10.3	14.2	39.2 µs	21/21	0.000
加权 A* (w=1.5)	17.8	27.1	10.3	14.2	50.0 µs	21/21	0.000
加权 A* (w=2.0)	17.7	27.0	10.4	14.4	49.6 µs	19/21	0.190

加权A*:f=g+wh 可以得出结论:曼哈顿距离优于dijkstra算法;w=1.5的搜索规模与不加权时相同,但用时更长;w=2时用时更短,但不能保证找到最优结果

2. 搜索顺序的影响:

搜索顺序	平均扩展节点数	平均路径步数	单次中位耗时
f 相同时先扩展 h 小者	18.5	14.1	43.9 µs
f 相同时按入队顺序	34.9	14.1	62.2 µs

结论:先扩展h小的节点可以缩小搜索规模,减少用时

3. 地图规模的影响: 将地图放大到40×40,更加明显的对比出算法之间的耗时差异

搜索策略	扩展节点数	单次中位耗时	最优解占比
h = 0（Dijkstra）	1183	1.849 ms	3/3
曼哈顿距离	155	0.601 ms	3/3

规模更大的地图,曼哈顿距离明显优于dijkstra.

题目2

算法实现

实现一个加权地形最短路径求解器,使用A*算法

A*状态: Coord = Tuple[int, int]

g:从起点到当前节点已知最小代价

h:启发式函数 h(n) = (d - 1) · m + min(m, e)

其中m是除终点外所有可通行格的最小代价,e是进入终点的代价;本题中公式退化为`h(n) = d - 1`.

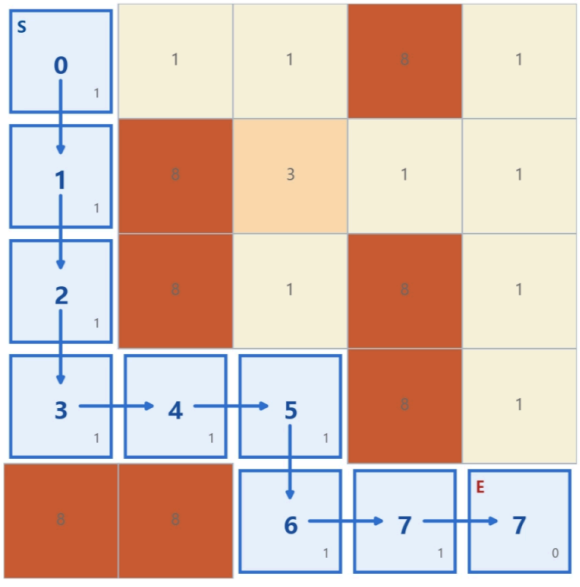
```
def heuristic(node):
    distance = abs(node[0] - goal[0]) + abs(node[1] - goal[1])
    if distance == 0:
        return 0.0
    return (distance - 1) * minimum + min(minimum, goal_cost)
```

可采纳性:路径总代价为$\sum_{i=1}^{k} \text{cost}(v_i)$,对于前k-1个节点,每个节点的代价都≥m,所以$\sum_{i=1}^{k} \text{cost}(v_i) \ge (k - 1)m + e \ge (d - 1)m + e$.若采用曼哈顿距离,对于临近终点的格子,其代价为0,而启发式函数值为1,会高估代价.

一致性:即h(n) ≤ c(n, n') + h(n'); 若n' = goal,此时 n 与 goal 相邻，故 d(n) = 1。于是h(n) = (1 - 1)m + M = M = min(m, e) ≤ e = cost(goal) = c(n, n')，且 h(goal) = 0，所以 h(n) - h(goal) = M ≤ e = c(n, n'); 若n' ≠ goal 且临近 goal, 此时必有 d(n) ≥ 2, 于是h(n) - h(n') = [(d - 1)m + M] - [(d - 2)m + M] = m.而 n' ≠ goal 且是合法邻居，所以 cost(n') ≥ m。因此h(n) - h(n') = m ≤ cost(n') = c(n, n') 若不属于以上2种情况,h(n) - h(n') = [(d - 1)m + M] - [dm + M] = -m ≤ 0 ≤ cost(n') = c(n, n')

f=g+h

算法步骤同题目1.



步	到达格	进入代价	累计 g
1	(1,0)	1	1
2	(2,0)	1	2
3	(3,0)	1	3
4	(3,1)	1	4
5	(3,2)	1	5
6	(4,2)	1	6
7	(4,3)	1	7
8	(4,4) = E	0	7

搜索效率评估

1. 不同启发函数的搜索规模与用时:

搜索策略	扩展节点数	生成节点数	队列峰值	路径步数	单次中位耗时	最优解占比	平均代价偏差
h = 0（Dijkstra 基线）	47.8	50.5	9.3	14.0	79.1 μs	25/25	0.000
曼哈顿距离 h	44.4	50.2	10.6	14.1	77.7 μs	25/25	0.000
可采纳地形启发函数	44.6	50.2	10.6	14.1	96.8 μs	25/25	0.000
加权 A* (w=1.5)	43.0	49.9	11.3	14.1	94.4 μs	25/25	0.000
加权 A* (w=2.0)	40.2	48.6	11.8	14.1	90.1 μs	25/25	0.000

所有方法都能求得最优解;加权A*算法的搜索规模小,但优先队列储存节点最大,所以单位耗时长

2. 地图规模对搜索性能的影响

搜索策略	扩展节点数	单次中位耗时	最优解占比
h = 0（Dijkstra）	1285	2.244 ms	3/3
曼哈顿距离	1282	2.394 ms	3/3
可采纳地形启发函数	1282	2.763 ms	3/3

加权地形上，按最小格点代价 1 缩放的曼哈顿距离几乎不提供信息量：扩展节点数与 Dijkstra 基本相同，启发函数自身的计算开销还会让耗时略增。可采纳只保证最优性，不保证信息量。

题目3

算法实现

minimax算法: Minimax 算法采用递归和深度优先搜索的方式遍历博弈树，从叶子节点开始向根节点逐层回传评估值。对于叶子节点，直接返回其评估值；对于 MAX 节点，选择所有子节点中的最大值作为当前节点的回传值；对于 MIN 节点，选择所有子节点中的最小值作为当前节点的回传值。经过逐层计算，最终得到根节点的最优决策值。

```
def _minimax(node, is_maximizing, stats):
    stats.nodes_visited += 1
    if not node.children:
        stats.leaves_evaluated += 1
        return float(node.value)
    values = [_minimax(child, _child_turn(child, is_maximizing), stats) for child in node.children]
    return max(values) if is_maximizing else min(values)
```

α-β 剪枝算法:

Alpha-Beta 剪枝是在 Minimax 算法基础上引入 α 和 β 两个边界参数。α 表示 MAX 当前已知的最低保证值，β 表示 MIN 当前已知的最高允许值。 在 MAX 节点中，当当前节点的值大于或等于 β 时，说明该分支不可能被上层 MIN 节点选择，因此可以停止搜索剩余子节点。 在 MIN 节点中，当当前节点的值小于或等于 α 时，说明该分支不可能被上层 MAX 节点选择，因此可以停止搜索剩余子节点。 当 α ≥ β 时，满足剪枝条件。通过跳过不会影响最终决策的分支，Alpha-Beta 算法能够减少节点访问数量，同时保证最终计算结果与 Minimax 算法一致。

```
def alpha_beta(node: TreeNode, is_maximizing: bool = True,
               alpha: float = NEG_INF, beta: float = POS_INF) -> SearchOutcome:
    stats = _Stats()
    value = _alpha_beta(node, is_maximizing, alpha, beta, stats)
    return SearchOutcome(value, stats.nodes_visited, stats.leaves_evaluated, stats.cutoffs)
```

剪枝效率评估

访问顺序对开销的影响

子节点访问顺序	平均访问节点数	平均求值叶节点数	平均剪枝次数
原始顺序（左→右）	25.0	11.6	2.9
完美排序（最有利在前）	18.0	7.0	5.0
最差排序（最有利在后）	30.7	15.8	0.2

使用最有利节点在前的访问顺序可以减少访问节点数量,增加剪枝数量,从而减少开销

子节点访问顺序对剪枝的影响

数据集	算法	访问节点数	求值叶节点数	剪枝次数	剪掉的子树节点数	中位耗时	搜索结果
题目给定用例	Minimax	15	8	0	0	3.7 µs	5
题目给定用例	Alpha-Beta	14	7	1	1	4.3 µs	5
随机 3 层树（30 例均值，27 叶）	Minimax	40.0	27.0	0.0	0.0	8.4 µs	-
随机 3 层树（30 例均值，27 叶）	Alpha-Beta	30.8	19.3	3.1	9.2	8.9 µs	-
随机 5 层树（10 例均值，243 叶）	Minimax	364.0	243.0	0.0	0.0	76.1 µs	-
随机 5 层树（10 例均值，243 叶）	Alpha-Beta	191.8	113.1	28.1	172.2	56.2 µs	

在所有情况下,α-β剪枝都能提升搜索效率,随着搜索树规模增加,提升情况更加明显.

题目4

算法实现

先把 A、B、T1、T2、T3、T4 这 6 个位置的两两距离全部用 A* 算法算出来，保存成 6×6 的矩阵；之后博弈搜索里，任何一步的代价都可以查表,为O(1)复杂度.

1. A*算法计算代价矩阵 建立一个6×6的矩阵,保存 A、B、T1、T2、T3、T4 这 6 个位置的两两距离.一共有*$C(6,2) = 15$ *对组合,对于每个组合使用A*计算代价.结果:

起点 \ 终点	A	B	T1	T2	T3	T4
A	0	12	10	4	8	8
B	12	0	8	8	4	10
T1	10	8	0	6	6	12
T2	4	8	6	0	6	6
T3	8	4	6	6	0	6
T4	8	10	12	6	6	0

2. 博弈搜索过程

记录当前状态:

```
class GameState:
    anchors: Tuple[str, str]      # A 所在实体, B 所在实体
    remaining: Tuple[str, ...]    # 剩余宝藏
    turn: int                     # 0=A(MAX), 1=B(MIN)
    score: Tuple[int, int]        # 累计收益
```

minimax搜索: 设 `V(state, d)` 是当前玩家在剩余深度 d 下的最优值:

```
d == 0 或 remaining == null:   V = Score_A - Score_B
turn == 0:                     V = max over 所有可选宝藏
turn == 1:                     V = min over 所有可选宝藏
```

α-β剪枝:

- 在 MAX 层，如果某个子节点的返回值 ≥ β，说明 MIN 绝不会让它走到这里，剪掉后面的兄弟节点;
- 在 MIN 层，如果返回值 ≤ α，说明 MAX 不会选这个分支，同样剪掉。

```
if value >= beta: break    # MAX 层
if value <= alpha: break   # MIN 层
```

运行结果:

```
H = 2  Minimax    => {"best_target": "T2", "best_value": -1}   访问 17 个节点
H = 2  Alpha-Beta => {"best_target": "T2", "best_value": -1}   访问 13 个节点
       主变例: [('A', 'T2', 2), ('B', 'T3', 3)]
H = 4  Minimax    => {"best_target": "T3", "best_value": 3}   访问 65 个节点
H = 4  Alpha-Beta => {"best_target": "T3", "best_value": 3}   访问 47 个节点
       主变例: [('A', 'T3', -1), ('B', 'T1', 1), ('A', 'T2', 0), ('B', 'T4', -5)]
```

搜索性能差异

搜索深度	算法	访问节点数	求值次数	剪枝次数	剪掉的着法数	中位耗时	最优首步	最优收益差
H=1	Minimax	5	4	0	0	6.3 µs	T2	2
H=1	Alpha-Beta	5	4	0	0	6.6 µs	T2	2
H=2	Minimax	17	12	0	0	21.3 µs	T2	-1
H=2	Alpha-Beta	13	8	2	4	18.4 µs	T2	-1
H=3	Minimax	41	24	0	0	53.5 µs	T2	2
H=3	Alpha-Beta	26	13	5	7	39.4 µs	T2	2
H=4	Minimax	65	24	0	0	88.4 µs	T3	3
H=4	Alpha-Beta	47	16	5	6	70.8 µs	T3	3

① 更深的搜索会产生更大开销,但是也会带来更优的收益；② Alpha-Beta剪枝后减少开销,但是结果不改变；③ 剪枝比例不随深度单调变化。 H=3 节省了 37%的开销，H=4 反而只省了 28%。说明剪枝效果取决于实际碰到的数值和着法顺序；④ 不同的博弈深度会改变结果.H=2时搜索层数浅,不能找到最优解;H=4经历的完整搜索,可以找出最优解.

不同访问顺序的剪枝效果

着法排序策略（H=4）	平均访问节点数	平均剪枝次数	中位耗时
按编号顺序 T1→T4	47.0	5.0	72.1 µs
自己最想要的排在前面	50.0	2.0	81.0 µs
自己最不想要的排在前面	39.0	5.0	65.9 µs
随机顺序（20 组种子均值）	42.1	4.2	0.254 ms

说明:剪枝效果取决于实际碰到的数值和着法顺序.